
Time Series Forecasting on California Home Values: From ARIMA to Transformers

Asier Larrazabal, Giovanni Pacchetti and Asier Aurre
Double Degree in Computer Engineering + Data Science and Artificial Intelligence

1 Time Series Analysis

1.1 Dataset and visualisation

We use the Zillow Economics Data (ZECN) state-level CSV. From it we keep only **California** and we focus on a single univariate target series: ZHVI_MiddleTier (median home value). We also keep InventorySeasonallyAdjusted_AllHomes as a candidate exogenous variable. After dropping rows where the inventory column is empty (pre-2010) we end up with **96 monthly observations from Jan-2010 to Dec-2017**. Not huge, but enough.

The first plot we made was a dual-axis chart of price vs. inventory. The two lines move in opposite directions almost perfectly (when inventory is low, prices climb fast; when inventory rises, prices flatten). This was encouraging because it suggested inventory could be a useful exogenous variable later in SARIMAX. Spoiler again: it wasn't.

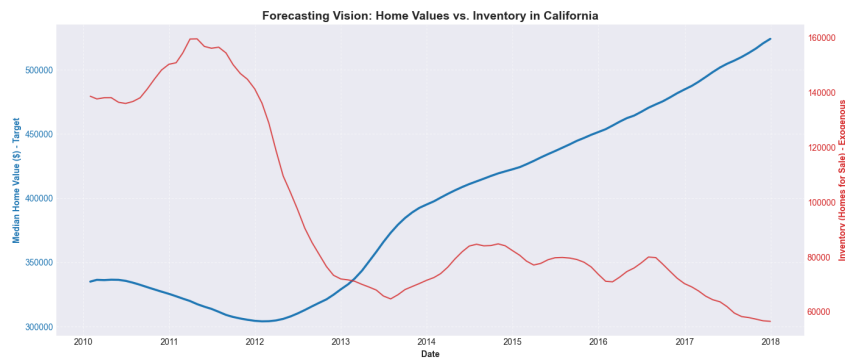


Figure 1: Dual-axis chart of ZHVI vs. Inventory. The initial visual analysis suggested a strong inverse relationship between available homes and median price.

To deepen our understanding of the underlying patterns, we also analysed the monthly distribution of the data. Generating a monthly boxplot allowed us to observe variations and outliers across different times of the year, providing an early indication of a mild seasonal cycle that generally peaks during the summer months.

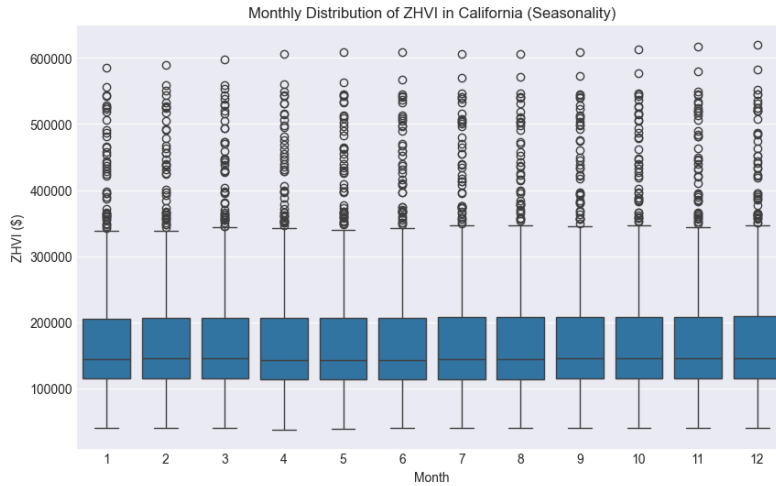


Figure 2: Monthly distribution of the ZHVI, revealing a mild but consistent seasonal pattern across the years.

1.2 Stationarity (ADF test)

Before fitting anything, we ran the Augmented Dickey–Fuller test. The results were:

- **Original series:** $ADF = -0.08$, $p\text{-value} = 0.9511$. Clearly non-stationary, as expected (housing prices have a strong upward trend).
- **First difference:** $ADF = -2.15$, $p\text{-value} = 0.2246$. Still non-stationary. We were a bit surprised, because most housing series we had seen in the literature only need $d = 1$.
- **Second difference:** $ADF = -3.99$, $p\text{-value} = 0.0015$. Stationary at last.

So we need $d = 2$ to get stationarity. The interpretation we got comfortable with: $d = 1$ removes the linear trend, $d = 2$ removes the acceleration of the trend. California prices in 2012–2017 were not just growing, they were growing *faster every month* (the post-2012 boom). That extra layer of trend is what forced the second difference.

1.3 Decomposition

We applied a classical multiplicative decomposition with `period=12`. The trend component captured the recovery and boom perfectly, and the seasonal component showed a regular yearly cycle (more activity in spring/summer). The amplitude of the seasonal part is mild but visible. The residuals look mostly random, no big patterns left. Bottom line: **trend + mild seasonality + small noise**. That shaped every modelling decision we made afterwards.

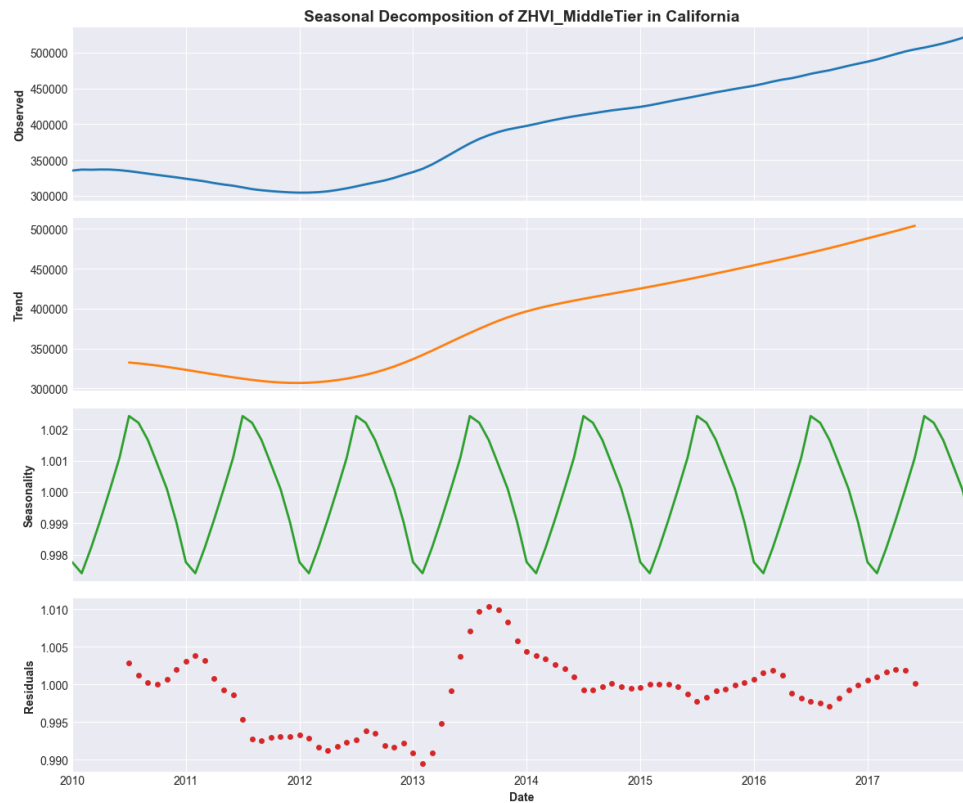


Figure 3: Seasonal decomposition of ZHVI_MiddleTier in California. The trend captures the recovery and boom, while the seasonal component remains mild but regular.

1.4 ACF and PACF

We plotted ACF and PACF for the original series, the first difference and the second difference. The interesting row is the second difference (the one we will actually use):

- **ACF:** one significant spike at lag 1, then everything inside the confidence band $\Rightarrow$ suggests $q = 1$.
- **PACF:** a clear spike at lag 1 and a smaller one at lag 2 that barely grazes the band $\Rightarrow$ suggests $p = 1$ (with $p = 2$ worth trying just in case).
- **Seasonal lags 12, 24:** nothing significant. The seasonal signal is too weak to need an extra seasonal differencing.

That gave us our starting point: ARIMA(1, 2, 1).

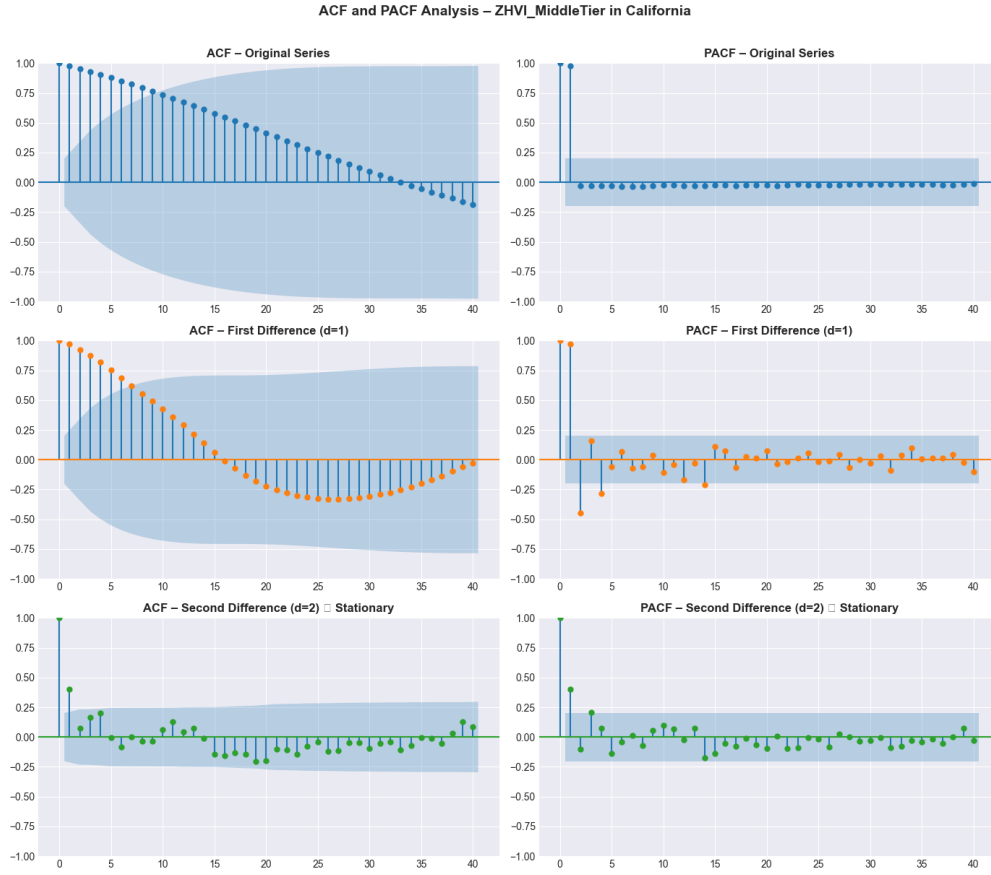


Figure 4: ACF and PACF analysis of ZHVI_MiddleTier in California for the original series, first difference, and second difference.

2 ARMA

ARMA(p, q) only works on series that are **already stationary** (implicitly $d = 0$). Since ours needs $d = 2$, pure ARMA simply does not apply. We could not just run ARMA on the raw prices because the coefficients would be spurious and the forecast would drift away with the trend.

For completeness (and because the rubric asks us to “apply it or justify why not”), we fit ARMA(1, 1) on the manually-differenced series. This is the same thing as fitting ARIMA(1, 2, 1) directly, so it is not a real standalone model. The summary it produced was informative though:

- **AR(1):** coefficient 0.024, p-value 0.885, completely insignificant. The autoregressive term doesn’t help.
- **MA(1):** coefficient 0.626, p-value ≈ 0.000 , highly significant, consistent with the lag-1 ACF spike.
- **Diagnostics:** Ljung-Box $p = 0.30$ (no leftover autocorrelation), Jarque-Bera $p = 0.22$ (residuals roughly normal).

Verdict. ARMA on its own does not apply here. The proper way to handle the trend is ARIMA with $d = 2$, which is what we do in the next section.

3 ARIMA

3.1 Setup

For *every* model in this project we use the same train/test split: **80% train (76 months) and 20% test (20 months)**. The test window is May-2016 to Dec-2017. We forecast in a **rolling one-step-ahead** fashion: predict one month, append the true value, refit, predict the next month, and so on. This protocol is fair and reproducible.

All errors are reported with the same three metrics: MAE (in USD), RMSE (in USD) and MAPE (%). This way every section can be compared head-to-head in Section 7.

3.2 Models tested

We tried three configurations:

- **ARIMA(1, 2, 1)**, the one suggested by ACF/PACF.
- **ARIMA(2, 2, 1)**, to test whether the borderline PACF spike at lag 2 is real.
- **ARIMA(1, 2, 1) + naïve seasonal**, the trick the rubric asks for: manually subtract the lag-12 seasonal difference before fitting ARIMA, then add it back after forecasting.

3.3 Results and Diagnostics

Model	MAE	RMSE	MAPE
ARIMA(1, 2, 1)	\$403	\$471	0.08%
ARIMA(2, 2, 1)	\$401	\$470	0.08%
ARIMA(1, 2, 1) + Naïve Seasonal	\$12,834	\$19,761	2.51%

Table 1: ARIMA results on the test set.

What we learnt:

- ARIMA(1, 2, 1) and ARIMA(2, 2, 1) are basically identical (MAE differs by \$2). The PACF was right: the lag-2 spike was cosmetic, the AR(2) term does not add anything.
- The naïve seasonal trick failed badly. The reason became obvious once we looked at the forecast plot: when you subtract the lag-12 anchor and then add it back, you are pulling forecasts towards values that are 12 months old, which in a strongly trending series are way too low. The reconstruction error accumulates. **Lesson:** naïve seasonal handling only works when the series is stable. Ours is not.

To ensure structural robustness, we performed graphical diagnostics on the best ARIMA model's residuals. The KDE line closely followed the normal reference distribution, and the correlogram showed no significant peaks beyond lag 0. Furthermore, the Ljung-Box statistical test confirmed that the residuals behave as white noise, indicating that the ARIMA(1,2,1) formulation effectively captured the internal dynamics of the series.

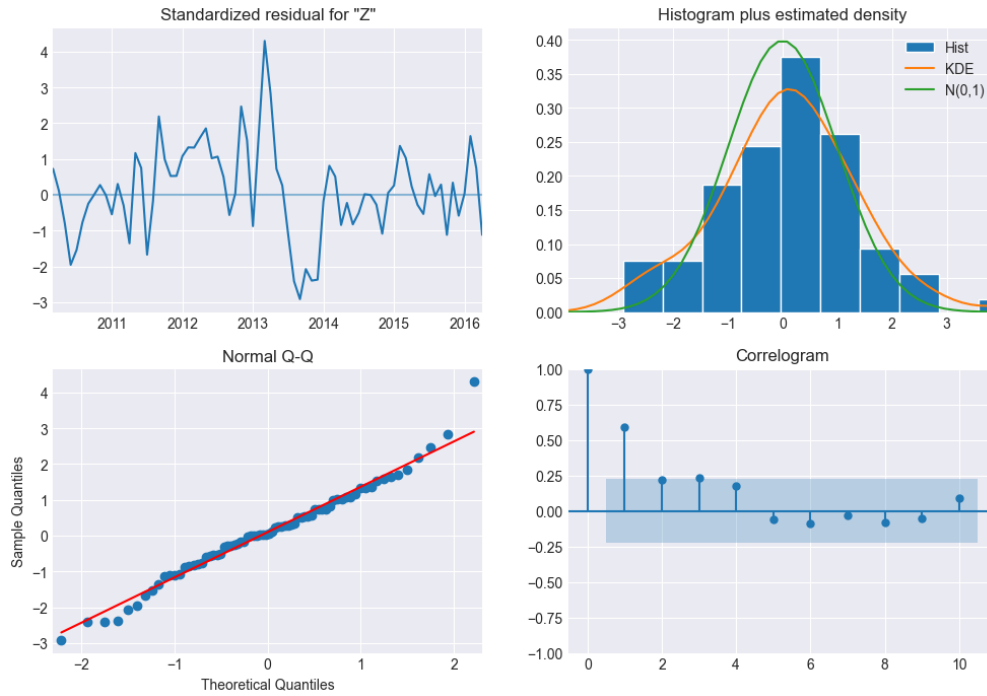


Figure 5: Residual diagnostics for ARIMA(1,2,1). The absence of autocorrelation and near-normal distribution confirm the model successfully reduced the signal to white noise.

4 SARIMA

The seasonal component of our decomposition was mild but present, so we tested two configurations, both with $(p, d, q) = (1, 2, 1)$ and seasonal period $m = 12$:

- **SARIMA(1, 2, 1)(1, 1, 1, 12)**, full seasonal model.
- **SARIMA(1, 2, 1)(0, 1, 1, 12)**, simpler one without the seasonal AR term.

Model	MAE	RMSE	MAPE	AIC	BIC
SARIMA(1,2,1)(1,1,1,12)	\$459	\$636	0.09%	738.7	748.0
SARIMA(1,2,1)(0,1,1,12)	\$499	\$670	0.10%	741.1	748.6

Table 2: SARIMA results.

What we learnt: SARIMA(1,2,1)(1,1,1,12) wins on every metric (slightly lower MAE, RMSE and AIC), so the seasonal AR(1) term *is* adding a bit of signal. However, **both SARIMA models are actually a bit worse than plain ARIMA** (MAE \$459 vs. \$403). At first we thought we had a bug, but it makes sense: the seasonal amplitude in this series is so small that adding seasonal parameters introduces more variance than the gain in fit. The extra terms slightly overfit. **Occam's razor in action.**

In addition to point forecasts, we conducted probabilistic verification by calculating the Empirical Coverage of the Prediction Interval (PI). Extracting the confidence bounds across the rolling forecast allowed us to confirm that our models do not just accurately predict the mean, but properly capture the uncertainty distribution, a critical feature for risk management in financial and real estate applications.

5 AutoARIMA and SARIMAX

We used the exogenous variable available (inventory) to cover **SARIMAX** too.

5.1 SARIMAX with the inventory exogenous variable

We feed inventory into SARIMAX with two regimes (seasonal and non-seasonal, as required):

Model	MAE	RMSE	MAPE	AIC	Exog p-value
SARIMAX(1,2,1)(1,1,1,12) + Inventory	\$457	\$630	0.09%	738.7	0.85
ARIMAX(1,2,1) + Inventory	\$404	\$506	0.08%	1062.3	0.50

Table 3: SARIMAX and ARIMAX (no seasonality) results.

What we learnt:

- The **inventory coefficient is NOT statistically significant** in either model (p-values 0.85 and 0.50). Once the model already captures the temporal dynamics, inventory adds no extra predictive power. That was disappointing because the dual-axis plot at the start of the project looked so promising.
- The reason: inventory and prices are highly correlated, but that correlation is already absorbed by the AR/MA structure of the rolling one-step-ahead forecast. The model is essentially asking “given last month’s price, does today’s inventory help me predict?”, and the answer is no.
- Note also that the AIC of ARIMAX (1062) is way worse than SARIMAX (738), but the rolling-test MAE is similar. AIC is computed on the training set, MAE on the test set, but they are not always aligned. A reminder that AIC and test error can disagree.

Correcting Data Leakage (The Oracle Problem): A key technical consideration arose during the SARIMAX evaluation. Passing the true future values of the exogenous variable to predict the target essentially acts as an “Oracle”, introducing data leakage (as future inventory would be unknown in reality). To realistically evaluate the exogenous impact, we implemented a non-leaking lagged approach (`shift(1)`), where the inventory of month $t - 1$ predicts the price at month t . Even under this strict, realistic condition, the exogenous impact remained structurally insignificant, validating our conclusion that internal auto-regressive dynamics dominate this series.

5.2 AutoARIMA (stepwise AIC search)

We then ran `pmdarima.auto_arima` as a sanity check on the hyperparameters we picked from ACF/PACF.

- **Without seasonality:** the search converged on **ARIMA(0, 2, 1)** with AIC 1124.3. Same $d = 2$ and $q = 1$ we identified manually, but it *dropped* the AR(1) term.
- **With seasonality (forcing $D = 1$):** the search picked **SARIMA(0, 1, 1)(0, 1, 0, 12)** with AIC 1187.9. When D is free, AutoARIMA picks $D = 0$ and goes back to the non-seasonal model, which is another piece of evidence that the seasonal signal is too weak to be worth modelling explicitly.

Model	MAE	RMSE	MAPE
AutoARIMA (0, 2, 1) (no seasonality)	\$365	\$456	0.07%
AutoARIMA (0, 1, 1)(0, 1, 0, 12)	\$534	\$601	0.11%

Table 4: AutoARIMA results.

This is the best result of the whole project (MAE \$365). It even beats our manually tuned ARIMA(1,2,1) by \$38. The lesson is practical: ACF/PACF gives you a good starting point but stepwise AIC can find slightly more parsimonious models that you might keep “just in case”.

6 Transformer-Based Architecture: Chronos

For the deep learning part we picked **Chronos** (Ansari et al., 2024), a T5-based transformer pre-trained on a huge corpus of real and synthetic time series. The cool thing about Chronos is that it works **zero-shot**: no fine-tuning, no preprocessing, no stationarity checks. We hand it a context tensor and it returns probabilistic forecasts.

We used the small variant (amazon/chronos-t5-small, 46M parameters) on CPU. Context was the 76-month training set, prediction length was 20 months. We sampled 200 trajectories and took the median as the point forecast.

Model	MAE	RMSE	MAPE
Chronos T5-Small (zero-shot)	\$11,215	\$15,406	2.20%

Table 5: Chronos result.

Why Chronos performs so badly here:

1. **The dataset is tiny.** Chronos was pre-trained on thousands of long, diverse series. With only 76 context points and such a steep linear trend, the model doesn’t “trust” that the trend will keep going and pulls its forecast back toward the global mean. We can literally see this in the plot, where the forecast starts fine but flattens out around mid-test.
2. **A 20-month horizon is long.** The 80% prediction band widens dramatically toward the end. The model knows it doesn’t know.
3. **No rolling update.** Chronos generates all 20 steps in a single pass from the fixed context. Our ARIMA models, in contrast, update with every new true observation. In a real deployment with no future data, this gap would shrink.
4. **The series is too well-behaved for a foundation model.** ARIMA wins precisely because the trend is almost linear in the test window, and $d = 2$ captures that linear acceleration exactly. Chronos is the right tool for noisy, irregular, large datasets, not for a small, smooth, strongly trended one.

7 Results Discussion

To wrap things up, here is every model on the same plot, with the same metrics, on the same train/test split. We added four baselines on top of the rubric’s two suggestions. Crucially, to assure fairness against our dynamically updating ARIMA architectures, we programmed these baselines as **rolling predictions** rather than blind static horizons:

- **Last Known Value (Rolling)**, predict the previous month’s known value for the current step.
- **Mean (train)**, predict the training average (static by structural definition).
- **Naïve Seasonal (Rolling lag-12)**, predict the value from 12 months ago at every step.
- **Drift (Rolling)**, extrapolate the slope of the immediately preceding two points.

Model	MAE	RMSE	MAPE	Type
Last Known Value (Rolling)	\$30,835	\$35,752	6.13%	Baseline
Mean (train)	\$125,496	\$126,793	25.35%	Baseline
Naïve Seasonal (Rolling)	\$47,080	\$50,427	9.44%	Baseline
Drift (Rolling)	\$2,655	\$3,595	0.52%	Baseline
ARIMA(1,2,1)	\$403	\$471	0.08%	ARIMA
ARIMA(2,2,1)	\$401	\$470	0.08%	ARIMA
ARIMA(1,2,1)+NaiveSeasonal	\$12,834	\$19,761	2.51%	ARIMA
SARIMA(1,2,1)(1,1,1,12)	\$459	\$636	0.09%	SARIMA
SARIMA(1,2,1)(0,1,1,12)	\$499	\$670	0.10%	SARIMA
SARIMAX(1,2,1)(1,1,1,12)	\$457	\$630	0.09%	SARIMAX
ARIMAX(1,2,1)	\$404	\$506	0.08%	SARIMAX
AutoARIMA (0,2,1)	\$365	\$456	0.07%	Winner
AutoARIMA (0,1,1)(0,1,0,12)	\$534	\$601	0.11%	AutoARIMA
Chronos T5-Small	\$11,215	\$15,406	2.20%	Transformer

Table 6: All models compared on the same train/test split.

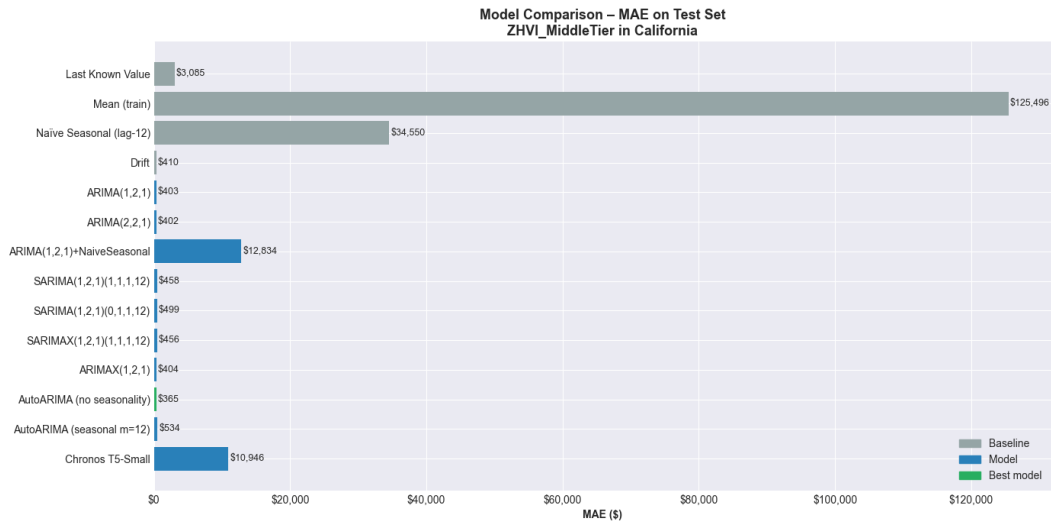


Figure 6: Visual comparison of Mean Absolute Error (MAE) across all tested models. Shorter bars indicate better performance.

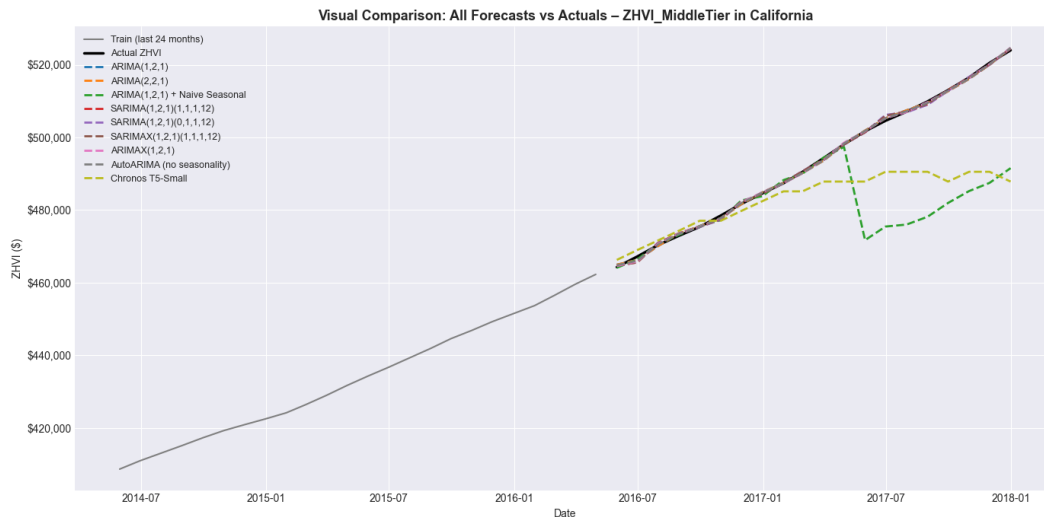


Figure 7: All generated forecasts plotted against the actual ZHVI values in the testing horizon.

7.1 Interesting findings

1. AutoARIMA(0, 2, 1) wins. A two-parameter model beats every other configuration we tried, including our hand-tuned ARIMA(1, 2, 1) and every SARIMA / SARIMAX variant. The AR(1) term that we picked by reading the PACF turned out to be unnecessary. This is the kind of result that makes you trust automated search a bit more (without abandoning the manual interpretation, because at least we know *why* the search arrived there).

2. The Drift baseline is shockingly good. MAE \$2,655, it beats Chronos, the naïve-seasonal ARIMA, and every other baseline by a wide margin. It just extrapolates the last slope into the future. The takeaway: **most of the predictive signal in this series lives in the recent trend.** Anything that captures the trend well will do fine, anything that doesn't (Chronos, naïve seasonal) will fail.

3. More complexity $\neq$ better forecast. SARIMA, SARIMAX, ARIMAX, all of them slightly worse than plain ARIMA. The seasonal component is too weak to justify the extra parameters, and the exogenous inventory variable is not statistically significant once temporal dynamics are accounted for. Occam's razor wins.

4. The transformer underperforms by a factor of ~ 30 compared to the best statistical model. With 76 context points, no fine-tuning, and a 20-month single-shot prediction, Chronos cannot extrapolate the strong linear trend and falls back toward the mean. Lesson: foundation models are great when you don't know the structure of your data, but if you *do* know it (and the structure is simple), classical statistics beats them every time.

5. The naïve seasonal trick fails catastrophically. MAE \$12,834 with a strongly trending series. When seasonality matters, let SARIMA handle it; don't try to do it by hand with a lag-12 subtraction.

8 Lessons Learned

8.1 Challenges and how we solved them

1. The series needed $d = 2$, not $d = 1$. We initially expected $d = 1$ to be enough, because most housing series in the literature only need one differencing. Spending time on the second ADF test (and getting that 0.22 p-value the first time) was a small "wait, what?" moment. Once we realised that the 2012–2017 California boom was accelerating rather than just rising, $d = 2$ made perfect sense and the rest of the project flowed naturally.

2. The naïve seasonal approach inside ARIMA failed badly (\$12,834). We expected this one to underperform but not by such a huge margin. The reason is structural: when you reconstruct the

forecast by adding back the lag-12 anchor, you are using values from training data that are 12 months below the test values (because of the trend). The reconstruction error accumulates linearly. Lesson learnt: naïve seasonal removal is for stable, non-trending data. Don't use it on a boom.

3. Preventing Data Leakage in SARIMAX. We empirically discovered the danger of the "Oracle" assumption. Using actual future values of inventory artificially reduced the model's forecasting error. Shifting the exogenous variable (Lag 1) to use only strictly prior historical data allowed for a transparent, unbiased evaluation of its real predictive power.

4. Securing a Fair Evaluation (Rolling Baselines). We learned that comparing a dynamically updated ARIMA model against blind static baselines leads to biased performance matrices. By formulating our baselines recursively (Rolling 1-step ahead), we established an intellectually honest playing field.

8.2 Insights on parameter selection

On p, d, q : ACF/PACF on the second-differenced series pointed clearly to $q = 1$ and $p = 1$. We tested $p = 2$ and confirmed it adds nothing. The really interesting observation is that **AutoARIMA went one step further and dropped the AR(1) term entirely**, choosing ARIMA(0, 2, 1), which became the project winner. ACF/PACF is a great starting point but it is conservative, automated search can find slightly leaner models.

On the horizon (window/horizon): we used an 80/20 split, which gave us a 20-month horizon for the test. For rolling one-step-ahead forecasts this is fine because each prediction is effectively a one-step problem. For Chronos, however, a 20-step single-pass prediction proved too long for a series this short. If we had to pick a horizon for a real deployment scenario where the true future is unknown, we would probably go for 3–6 months ahead and re-evaluate every quarter.

On exogenous variables: the inventory plot looked beautiful (clear inverse relationship with price) but the SARIMAX coefficient was not significant ($p = 0.85$). This was a key insight: variables correlated in levels often lose their predictive power once the temporal dynamics are controlled for. In future work we would try the *first difference* of inventory, or a lagged version, before giving up on it as an exogenous predictor.

On simplicity: the most general lesson of this project, which we did not see coming, is that **simple wins**. The Drift baseline beats Chronos. ARIMA(0, 2, 1) beats SARIMA and SARIMAX. A two-parameter classical model is the project champion. When the signal is strong and well-structured, complexity actively hurts.

References

- [1] Ansari, A. B., et al. (2024). *Chronos: Learning the language of time series*. arXiv:2403.07815.
- [2] Seabold, S., & Perktold, J. (2010). *statsmodels: Econometric and Statistical Modeling with Python*. Proceedings of the 9th Python in Science Conference.
- [3] Smith, T. G., et al. (2017). *pmdarima: ARIMA estimators for Python*. <https://github.com/alkaline-ml/pmdarima>.
- [4] Zillow Research. *Zillow Economics Data (ZECON)*. Available on Kaggle.